袁友保10232140406 Lab1

```c
int bitAnd(int x, int y) {
    /*
     * 布尔代数运算
     * 记 x 和 y 分别为一个位
     * (x & y) 等价于 ~(~(x & y)) 等价于 ~(~x | ~y)
     */

    return ~(~x | ~y);
}
int getByte(int x, int n) {
    /*
     * 使用右移运算符，将目标字节右移到最低位
     * 再与 0xff 做与运算
     * 仅获取最低字节的内容
     */

    return (x >> (n << 3)) & 0xff;
}
int logicalShift(int x, int n) {
    /*
     * 先右移，由于逻辑右移在左侧补 0
     * 故将高位补充的 1 或 0 都与 0 做与运算
     */
    int xx = x >> n;
    int y = xx & (((1 << (31 + ~n + 1)) << 1) + ~1 + 1);

    return y;
}
int bitCount(int x) {
    /*
     * 相邻两个位相加，即为每 2 个位一组的每个组中 1 的个数
     * 将上一行的结果，相邻两个 2 位数相加，可得每 4 个位一组，每组中 1 的个数
     * 将上一行的结果，相邻两个 4 位数相加，可得每 8 个位一组，每组中 1 的个数
     * 以此类推直到全部位中 1 的个数
     */
    int xx = x;
    int mask_1 = 0x55;
    int mask_2 = 0x33;
```

```
    int mask_4 = 0x0f;
    int mask_8 = 0xff;
    int mask_16 = 0xff;
    mask_1 = (mask_1 << 8) + mask_1;
    mask_1 = (mask_1 << 16) + mask_1;
    mask_2 = (mask_2 << 8) + mask_2;
    mask_2 = (mask_2 << 16) + mask_2;
    mask_4 = (mask_4 << 8) + mask_4;
    mask_4 = (mask_4 << 16) + mask_4;
    mask_8 = (mask_8 << 16) + mask_8;
    mask_16 = (mask_16 << 8) + mask_16;
    xx = (xx & mask_1) + ((xx >> 1) & mask_1);
    xx = (xx & mask_2) + ((xx >> 2) & mask_2);
    xx = (xx & mask_4) + ((xx >> 4) & mask_4);
    xx = (xx & mask_8) + ((xx >> 8) & mask_8);
    xx = (xx & mask_16) + ((xx >> 16) & mask_16);

    return xx;
}
int bang(int x) {
    /*
     * 注意到 0 与非 0 数的不同之处为
     * 0 与 -0 的最高位都为 0，而非 0 数 x 与 -x 中总有一个的最高位为 1
     * 这是由于非零负数用最高位表示符号位导致的
     * 故只需判断 x 与 -x 的最高位中是否有 1
     *
     * 下面有另一种解法
     * 具体是通过二分的方法获取到数字的二进制表示中是否含有 1
     * 含有 1 则输出 0
     * 否则输出 1
     */

    return (((x | (~x + 1)) >> 31) & 1) ^ 1;
}
/*
int bang(int x) {
    int a = (x >> 16) | x;
    int b = (a >> 8) | a;
    int c = (b >> 4) | b;
    int d = (c >> 2) | c;
    int e = (d >> 1) | d;

    return ((~e) & 1);
```

```
}
*/
int tmin(void) {
    /*
     * 以 2 的补码表示的32位整型的最小值为 -2147483648
     * 即 0x80000000
     */

    return 0x80 << 24;
}
int fitsBits(int x, int n) {
    /*
     * https://www.zybuluo.com/SovietPower/note/1790502#csapp-lab1
     * 有显示该函数可能无法通过
     * 本地测试确实如此
     * 报错包括 fitsBits(0, 32) should be 0 等
     *
     * 将数字左移 32 - n 位再右移 32 - n 位，如果数字没有变化，说明该数字用 n 位可以表示
     * 否则说明该数字用 n 位会发生溢出
     * 由于位移不能超过 31 位的情况，将位移分成 15 - n 位和 17 位两次
     */
    int mov1 = 16 + ~n;
    int mov2 = 17;

    return !(((((x << mov1) << mov2) >> mov1) >> mov2) ^ x);
}
int divpwr2(int x, int n) {
    /*
     * 补码除法的公式为 (x < 0 ? x + (1 << k) - 1 : x) >> k
     * 不能使用 if 语句判断是否为负数
     * 故通过令非负数的偏置值为 0 的方式来规避判断
     * 可以利用符号位和偏置值的与运算
     * 来使得非负数的偏置值为 0
     */
    int mask = (x >> 31);
    int bias = ((1 << n) + ~1 + 1) & mask;
    int res = (bias + x) >> n;

    return res;
}
int negate(int x) {
    /*
     * 一个数取非相当于这个数字取反后 + 1
```

```
     */

    return ~x+1;
}
int isPositive(int x) {
    /*
     * 一个数的正负取决于这个数的最高位
     * 通过右移 31 位获取数字的符号位
     * 一个数为正数当且仅当它的符号位不为 1 且它本身不为 0
     */
    int sign = (x >> 31) & 1;

    return !sign & !!x;
}
int isLessOrEqual(int x, int y) {
    /*
     * 求 x 是否小于等于 y
     * 只需求 y - x 是否大于等于 0
     * 当两数之差超过 TMAX 时会发生溢出
     * 故需要分两数同号、两数异号两种情况讨论
     * 当两数同号时作差不会溢出，直接判断差值的符号位
     * 当两数异号的作差可能溢出，直接判断是否是 x 为负且 y 为正
     */
    int dif = (y + ~x + 1);
    int sign_of_dif = (dif >> 31) & 1;
    int signx = (x >> 31) & 1;
    int signy = (y >> 31) & 1;
    int dif_sign = (signx ^ signy);

    return (dif_sign & signx) | (!dif_sign & !sign_of_dif);
}
int ilog2(int x) {
    /*
     * 对一个数求 log2
     * 相当于求这个数中最高位的 1 的位置
     * 使用二分折半寻找最高位的 1
     * 先获取高 16 位的值，如果不为 0 说明高 16 位中有 1
     * 将结果 + 16，并将高 16 位移位至低 16 位，寻找此时低 16 中位最高位的 1
     * 如果为 0 说明最高位的 1 在低 16 位中，则不做操作，直接在低 16 位中寻找
     * 对现在的低 16 位数字，获取高 8 位的值，做同上的操作
     * 逐次迭代，寻找到最高位的 1 的位置
     * 即 ilog2 的答案
     */
```

```
    int res = 0;
    res = res + (!!(x >> 16) << 4);
    x = x >> (!!(x >> 16) << 4);
    res = res + (!!(x >> 8) << 3);
    x = x >> (!!(x >> 8) << 3);
    res = res + (!!(x >> 4) << 2);
    x = x >> (!!(x >> 4) << 2);
    res = res + (!!(x >> 2) << 1);
    x = x >> (!!(x >> 2) << 1);
    res = res + (!!(x >> 1) << 0);

    return res;
}
unsigned float_neg(unsigned uf) {
    /*
     * 浮点数的符号只由最高位直接决定
     * 分别获取阶码位和尾数位
     * 如果阶码位为全 1 且尾数为不为 0
     * 说明时 NaN，直接返回
     * 否则说明为数值，包括 0, -0, oo, -oo, 和规格化数
     * 此时只需将最高位的符号位取反，就是结果
     */
    unsigned res = uf;
    unsigned mask_significand = -1U >> 9;
    unsigned mask_exponent = 0xff << 23;
    unsigned exponent = mask_exponent & uf;
    unsigned significand = mask_significand & uf;
    if(exponent == mask_exponent && significand)
        res = uf;
    else
        res = uf ^ (1 << 31);

    return res;
}
unsigned float_i2f(int x) {
    /*
     * 根据浮点数的表示规则
     * 分别计算符号位、阶码、尾数
     * 符号位直接获取负号
     *
     * 先计算尾数
     *
     * 尾数通过将原数移位，使其最高位的 1 在第 24 位（从 1 开始计数）
```

```
 * （由于 implied leading 1 的存在，最高位的 1 将会舍弃，剩余的 1 ~ 23 位即为尾数）
 * 设原数最高位的位置为 k
 * 当 k 超过 24 时，可能会有舍入的情况存在
 * 当 k & (-1U >> 56 - k) > (1 << k - 25) 时，属于大于中间数的情况，应当进位，即在结果中 +
 * 当 k & (-1U >> 56 - k) < (1 << k - 25) 时，属于小于中间数的情况，应当直接舍去
 * 当恰好等于时，属于中间值的情况，应判断被舍入部分的高 1 位，如果是 1 说明舍去后是奇数
 * 要保证向偶数舍入应进位，即在结果中 + 1
 * 如果高 1 位是 0 说明舍去后是偶数，直接舍去
*
 * 再计算阶码
*
 * 只需求原数的 k，k - 1 就是阶码的值
 * 使用 E = k - 1 = e - 127
 * e = E + 127 = k - 1 + 127 = k + 126 即为阶码位
 *
 * 将各个部分相加得出结果
 * 当输入为 0 时，直接返回 0
 */
int high_bit = 0;
unsigned exponent = 0;
unsigned sign = 0;
unsigned significand = 0;
unsigned tmpx = 0;
unsigned res = 0;
unsigned mask_roundF = 0;
unsigned mask_round1 = 0;
if(x < 0){
    x = -x;
    sign = 1 << 31;
}
tmpx = x;
while(tmpx){
    high_bit++;
    tmpx = tmpx >> 1;
}
mask_roundF = -1U >> (55 - high_bit);
mask_round1 = 1 << (high_bit - 25);
if(high_bit >= 25){
    if((x & (mask_roundF >> 1)) > (mask_round1))
        res = 1;
    else if((x & mask_roundF) == ((mask_round1 << 1) + mask_round1))
        res = 1;
}
```

```
    significand = ((x << (32 - high_bit)) >> 8);
    significand = significand & (-1U >> 9);
    exponent = (high_bit + 126) << 23;
    res = res + exponent + sign + significand;
    if(x == 0)
        res = 0;

    return res;
}
unsigned float_twice(unsigned uf) {
    /*
     * 如果 uf 是 +0 或 -0，那么 2  * uf 就是它本身
     * 如果阶码为 0xff，则它是特殊值，同样返回本身
     * 如果阶码为 0，则它是非规格化数，阶码值固定为 1 - Bias
     * 而尾数不足 1，将其乘 2 只需将尾数乘 2
     * 即将尾数左移 1 位
     * 否则
     * 由于规格化浮点数通过 (1 + frac)  * 2 ^ k 来表示
     * 将浮点数乘 2
     * 只需将 k 值 + 1
     * 即将阶码值 + 1 后返回
     */
    unsigned res = 0;
    unsigned sign = uf & (1 << 31);
    unsigned exp = (uf >> 23) & 0xff;
    if(uf == 0 || uf == (1 << 31) || exp == 0xff)
        res = uf;
    else if(exp == 0)
        res = (uf << 1) | sign;
    else
        res = uf + (1 << 23);

    return res;
}
```